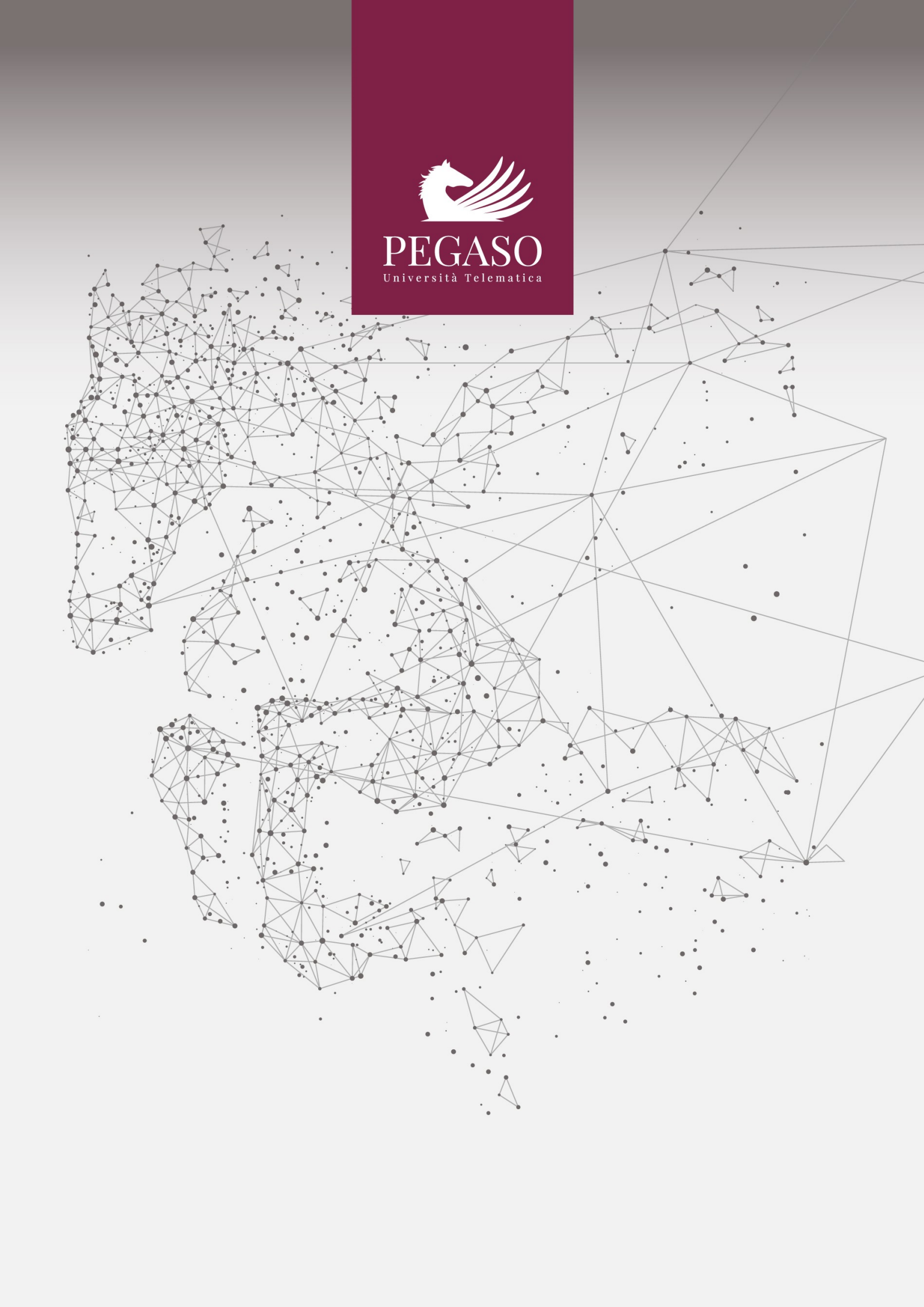

PEGASO
Università Telematica

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

Indice

1. ANGULARJS 3
2. ESEMPI 5
3. EVENTI 9
4. FORM 12
5. HTTP SERVICE 14
BIBLIOGRAFIA 16
SITOGRAFIA 17

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

1. Angularjs

Angular è un framework per lo sviluppo di applicazioni web basate su JavaScript/TypeScript. Creato da Google, Angular fornisce un insieme di strumenti e librerie per semplificare lo sviluppo di applicazioni web complesse e scalabili.

Angular utilizza il pattern architetturale MVC (Model-View-Controller) per organizzare il codice e semplificare la gestione della logica dell'applicazione:

- il Model rappresenta i dati dell'applicazione;
- la View è responsabile della presentazione dei dati;
- il Controller, chiamato "Component", gestisce la logica dell'applicazione e la comunicazione tra Model e View.

Angular utilizza anche i servizi per gestire la logica di business dell'applicazione e le dipendenze tra i diversi componenti.

Tra le principali caratteristiche di Angular ci sono:

- **Modularità**
- **Binding dei dati**
- **Routing**
- **Testing**
- **Animazioni**

Angular è scritto in TypeScript, un linguaggio di programmazione basato su JavaScript che fornisce una serie di funzionalità aggiuntive, come la tipizzazione statica e la gestione delle classi. TypeScript viene poi compilato in JavaScript, che può essere eseguito dai browser web moderni.

La CDN (Content Delivery Network) di AngularJS è un servizio che permette di includere la libreria AngularJS all'interno della propria pagina web tramite un URL fornito dal provider della CDN, senza dover scaricare la libreria e installarla localmente.

Per utilizzare la CDN di AngularJS, è sufficiente includere il seguente URL nella pagina HTML:

```
<script
src="https://ajax.googleapis.com/ajax/libs/angularjs/1.8.2/angular.min.js
"></script>
```

In questo modo, la libreria AngularJS verrà caricata dal server della CDN di Google.

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

SCOPE

Un'applicazione AngularJS può essere suddivisa in tre parti principali:

- la **vista** (HTML)
- il **modello** (dati disponibili per la vista)
- il **controller** (funzioni JavaScript che modificano il modello).

Lo **scope** è l'oggetto JavaScript che rappresenta il modello dell'applicazione in AngularJS: contiene le proprietà e i metodi che sono disponibili per la vista e il controller: lo **scope** consente al controller di definire le proprietà e i metodi che saranno utilizzati dalla vista, e di interagire con il modello per modificare i dati visualizzati dall'utente.

D'altra parte, la vista può utilizzare le proprietà e i metodi dello **scope** per visualizzare e interagire con i dati dell'applicazione: in questo modo, lo **scope** funge da tramite tra la vista e il controller, consentendo di creare applicazioni web più dinamiche e interattive.

```
<div ng-app="myApp" ng-controller="myCtrl">
<h1>{{carname}}</h1>
</div>

<script>
var app = angular.module('myApp', []);
app.controller('myCtrl', function($scope) {
  $scope.carname = "Ferrari";
});
</script>
```

Quando si aggiungono proprietà all'oggetto $scope nel controller di AngularJS, la vista può accedere a queste proprietà; nella vista, non è necessario utilizzare il prefisso $scope, ma è sufficiente fare riferimento al nome della proprietà: ad esempio, se abbiamo definito la proprietà $scope.carname nel controller, possiamo utilizzare la sintassi di interpolazione delle stringhe {{ carname }} o la direttiva ng-bind="carname" nella vista per visualizzare il valore della proprietà.

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

2. Esempi

All'interno del body possiamo in realtà "ospitare" più ng-controllers:

```
<div ng-controller="myCtrl">
    <h1>{{ greeting }}</h1>
      <input type="text" ng-model="name">
    <p>Hello, {{ name }}!</p>
  </div>
  <div ng-controller="userController">
    <h1>Users</h1>
    <ul>
      <li ng-repeat="user in users">
        {{ user.name }} ({{ user.email }})
      </li>
    </ul>
  </div>
```

Ed associare comportamenti all'interno del file javascript:

```
app.controller('userController', function($scope, $http) {

$http.get('https://jsonplaceholder.typicode.com/users').then(function(res
ponse) {
    $scope.users = response.data;
  });
});
```

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

In questo caso l'output sarà del tipo:



Figura 1 *- ng-controller*

Analizziamo alcuni esempi "base":

```
<div ng-app="" ng-init="firstName='John'">
<p>The name is <span ng-bind="firstName"></span></p>
</div>
```

In questo esempio, la direttiva ng-init viene utilizzata per inizializzare la variabile $scope.firstName con il valore 'John'; la direttiva ng-bind viene utilizzata per legare il valore della variabile $scope.firstName all'elemento HTML , in modo che il nome venga visualizzato nella pagina.

Consideriamo un nuovo esempio:

```
app.controller('myController', function($scope, $http) {
  var username = "octocat"; // inserisci qui il nome utente di GitHub
  $http.get('https://api.github.com/users/' + username +
'/repos').then(function(response) {
    $scope.repos = response.data;
  });
});
```

In questo esempio, abbiamo utilizzato l'API pubblica di GitHub per recuperare una lista di repository di un utente specificato tramite la variabile username. Quando la risposta viene ricevuta dal server, la lista di repository in formato JSON viene assegnata alla proprietà $scope.repos.

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

Per visualizzare i repository nella pagina HTML, possiamo utilizzare la direttiva ng-repeat per iterare sull'array di repository e visualizzarne il contenuto, ad esempio nel seguente modo:

```
<div ng-controller="myController">
    <h1>REPOS</h1>
    <ul>
      <li ng-repeat="repo in repos"><a href="{{ repo.html_url }}">{{
repo.name}}</a></li>
    </ul>
  </div>
```

La direttiva ng-repeat è utilizzata per ripetere un elemento HTML per ogni elemento di una collezione, come ad esempio un array:

```
<div ng-app="" ng-init="names=['Jani','Hege','Kai']">
  <ul>
    <li ng-repeat="x in names">
      {{ x }}
    </li>
  </ul>
</div>
```

Nell'esempio fornito, la direttiva ng-repeat viene utilizzata per iterare sull'array names e generare un elemento <li> per ogni elemento dell'array. La sintassi ng-repeat="x in names" indica che la variabile x rappresenta l'elemento corrente dell'array durante l'iterazione.

In questo esempio, l'array names contiene semplici stringhe di testo, che vengono visualizzate nell'HTML utilizzando la sintassi di interpolazione delle stringhe {{ }}.

Analizziamo ora il seguente esempio:

```
<div ng-app="" ng-init="names=[
{name:'Jani',country:'Norway'},
{name:'Hege',country:'Sweden'},
{name:'Kai',country:'Denmark'}]">

<ul>
  <li ng-repeat="x in names">
    {{ x.name + ', ' + x.country }}
  </li>
</ul>

</div>
```

In questo esempio, l'array names contiene oggetti con proprietà name e country. Utilizzando la sintassi {{ x.name + ', ' + x.country }}, la direttiva ng-repeat visualizza il nome e il paese di ogni oggetto nella lista.

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

AngularJS fornisce direttive che consentono di legare i dati dell'applicazione agli attributi degli elementi DOM HTML. La direttiva ng-disabled lega i dati dell'applicazione all'attributo disabled degli elementi HTML: questa direttiva viene utilizzata per disabilitare o abilitare gli elementi HTML in base al valore di un'espressione.

Ad esempio, supponiamo di avere un elemento di pulsante nella nostra applicazione AngularJS: possiamo utilizzare la direttiva ng-disabled per disabilitare il pulsante se un'espressione specificata nel controller restituisce true. Ecco un esempio di codice HTML che utilizza la direttiva ng-disabled:

```
<button ng-disabled="isDisabled">Pulsante</button>
```

In questo esempio, la direttiva ng-disabled è legata all'espressione isDisabled, che è definita all'interno del controller. Se l'espressione restituisce true, il pulsante verrà disabilitato; al contrario, se l'espressione restituisce false, il pulsante sarà abilitato.

```
app.controller('MyController', function($scope) {
  $scope.isDisabled = false;

  $scope.submitForm = function() {
    $scope.isDisabled = true;
    // effettua qui una richiesta HTTP o altre operazioni asincrone
    // quando l'operazione è completata, imposta isDisabled a false per
abilitare nuovamente il pulsante
  };
});
```

La direttiva ng-disabled è particolarmente utile per disabilitare gli elementi HTML durante l'elaborazione di un'operazione asincrona, come ad esempio l'invio di una richiesta HTTP al server: in questo caso, la direttiva ng-disabled può essere utilizzata per impedire all'utente di interagire con l'interfaccia utente durante l'attesa della risposta dal server.

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

3. Eventi

In AngularJS, gli eventi rappresentano l'interazione dell'utente con l'interfaccia utente, come ad esempio un click del mouse, una pressione di tasti o la selezione di una casella di testo. Gli eventi consentono di rendere le applicazioni AngularJS più interattive e reattive, poiché consentono di eseguire del codice in risposta all'interazione dell'utente.

AngularJS fornisce le proprie direttive per gestire gli eventi HTML. Queste direttive possono essere utilizzate per aggiungere ascoltatori di eventi a elementi HTML e per eseguire del codice in risposta a tali eventi.

Le direttive degli eventi in AngularJS sono:

- ng-blur
- ng-change
- ng-click
- ng-copy
- ng-cut
- ng-dblclick
- ng-focus
- ng-keydown
- ng-keypress
- ng-keyup
- ng-mousedown
- ng-mouseenter
- ng-mouseleave
- ng-mousemove
- ng-mouseover
- ng-mouseup
- ng-paste

Queste direttive possono essere utilizzate per associare del codice alla gestione degli eventi di un elemento HTML.

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

Ecco come utilizzare la direttiva ng-click per eseguire del codice quando un pulsante viene cliccato in AngularJS:

```
<button ng-click="showMessage()">Cliccami!</button>
```

In questo esempio, abbiamo creato un pulsante HTML e abbiamo aggiunto la direttiva ng-click ad esso: l'espressione showMessage() è associata alla direttiva ng-click e viene eseguita quando l'utente fa clic sul pulsante:

```
app.controller('MyController', function($scope) {
  $scope.message = "Ciao mondo!";

  $scope.showMessage = function() {
    alert($scope.message);
  };
});
```

In questo esempio, abbiamo definito un controller chiamato MyController e abbiamo assegnato alla variabile $scope.message il valore "Ciao mondo!". Successivamente, abbiamo definito una funzione chiamata showMessage, che viene eseguita quando l'utente fa clic sul pulsante; all'interno della funzione showMessage, utilizziamo la funzione alert per visualizzare il messaggio memorizzato nella variabile $scope.message.

In sintesi, quando l'utente fa clic sul pulsante, viene eseguita la funzione showMessage del controller, che visualizza un messaggio di alert contenente il valore della variabile $scope.message.

La direttiva ng-click può essere utilizzata per associare del codice alla gestione degli eventi di click di qualsiasi elemento HTML. Quando l'evento viene attivato, il codice associato viene eseguito, consentendo di effettuare azioni specifiche in risposta all'interazione dell'utente.

Introduciamo ora il codice per gestire un evento di mouse move ed over:

```
<!DOCTYPE html>
<html>
<script
src="https://ajax.googleapis.com/ajax/libs/angularjs/1.6.9/angular.min.js
"></script>
<body>

<div ng-app="myApp" ng-controller="myCtrl">

<h1 ng-mousemove="count_move = count_move + 1">Mouse Move!</h1>

<h2>{{ count_move }}</h2>

<h1 ng-mouseover="count_over = count_over + 1">Mouse Over!</h1>
```

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

```
<h2>{{ count_over }}</h2>

</div>
<script>
var app = angular.module('myApp', []);
app.controller('myCtrl', function($scope) {
    $scope.count_move = 0;
    $scope.count_over = 0;
});
</script>

</body>
</html>
```

La direttiva ng-mousemove è associata all'elemento h1 e viene eseguita ogni volta che l'utente muove il mouse sopra di esso. All'interno della direttiva, incrementiamo la variabile count_move di 1 ogni volta che l'utente muove il mouse sopra l'elemento.

La direttiva ng-mouseover, invece, è associata all'elemento h1 e viene eseguita ogni volta che l'utente posiziona il mouse sopra di esso. All'interno della direttiva, incrementiamo la variabile count_over di 1 ogni volta che l'utente posiziona il mouse sopra l'elemento.

Infine, abbiamo aggiunto due elementi h2 per visualizzare il conteggio delle volte in cui l'utente muove il mouse sopra e sopra l'elemento HTML, rispettivamente.

In sintesi, questo codice utilizza le direttive ng-mousemove e ng-mouseover per contare il numero di volte in cui l'utente muove il mouse sopra e sopra un elemento HTML, e visualizza il risultato utilizzando la sintassi delle doppie parentesi graffe di AngularJS.

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

4. Form

AngularJS fornisce molti strumenti per semplificare la gestione delle form HTML. Ecco alcuni esempi:

- **Validazione delle form**: AngularJS consente di validare facilmente i campi di input delle form. Utilizzando la direttiva ng-model, è possibile associare un campo di input a una variabile del controller. Inoltre, AngularJS fornisce molte direttive per la validazione dei campi di input, come ng-required, ng-pattern, ng-minlength, ng-maxlength e altre ancora.
- **Aggiornamento della form in tempo reale**: grazie alla bidirezionalità dei dati di AngularJS, la form si aggiorna in tempo reale mentre l'utente digita nel campo di input. Ciò significa che non è necessario aggiornare manualmente la form quando l'utente inserisce i dati.
- **Reset della form**: AngularJS fornisce una direttiva ng-reset per reimpostare i valori di tutti i campi della form.
- **Submit della form**: AngularJS consente di gestire la sottomissione della form utilizzando la direttiva ng-submit. Quando l'utente fa clic sul pulsante di submit, la funzione del controller associata alla direttiva ng-submit viene eseguita e può elaborare i dati della form.
- **Messaggi di errore**: AngularJS consente di visualizzare i messaggi di errore associati ai campi di input che non soddisfano i criteri di validazione. Ciò significa che l'utente può vedere immediatamente dove si trovano gli errori e correggerli.
- **Form dinamiche**: AngularJS consente di creare form dinamiche aggiungendo o rimuovendo campi di input in base alle esigenze dell'utente. Questo può essere utile quando si creano form complesse con molti campi.

Ecco un esempio di come utilizzare le direttive di validazione di AngularJS per validare una form HTML:

```
<form name="myForm" ng-controller="MyController" ng-submit="submitForm()" novalidate>
  <div>
    <label for="name">Nome:</label>
    <input type="text" name="name" ng-model="user.name" ng-required="true">
    <div ng-show="myForm.name.$error.required">Il nome è obbligatorio.</div>
  </div>

  <div>
    <label for="email">Email:</label>
    <input  type="email"  name="email"  ng-model="user.email"  ng-required="true"  ng-
pattern="/^.+@.+\..+$/">
    <div ng-show="myForm.email.$error.required">L'email è obbligatoria.</div>
    <div ng-show="myForm.email.$error.pattern">L'email non è valida.</div>
```

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

```
  </div>

  <button type="submit" ng-disabled="myForm.$invalid">Invia</button>

</form>
```

In questo esempio, abbiamo creato una form HTML con due campi di input (nome e email) e un pulsante di submit. Abbiamo anche utilizzato le direttive ng-required e ng-pattern per validare i campi di input. La direttiva ng-show viene utilizzata per mostrare i messaggi di errore associati ai campi di input invalidi.

```
app.controller('MyController', function($scope) {
  $scope.user = {};

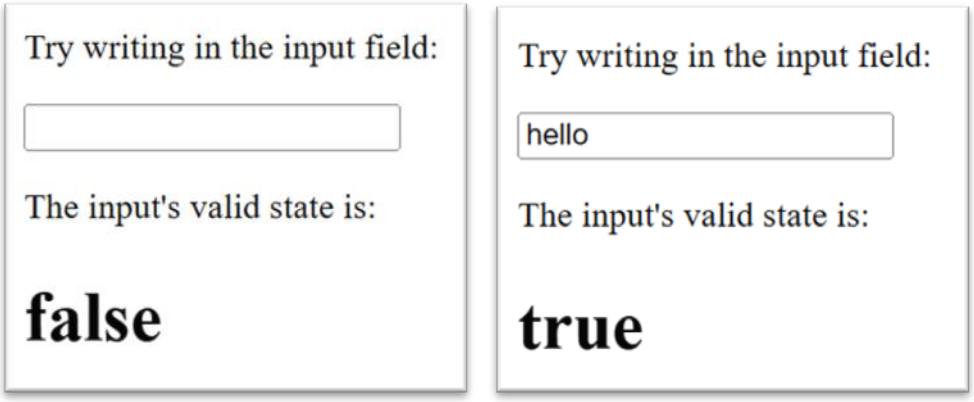
  $scope.submitForm = function() {
    alert("La form è stata inviata!");
  };
});
```

In questo esempio, abbiamo definito un controller chiamato MyController e abbiamo definito una variabile "user" per memorizzare i dati della form. Abbiamo anche definito una funzione chiamata submitForm, che viene eseguita quando l'utente fa clic sul pulsante di submit. All'interno della funzione, utilizziamo la funzione "alert" per visualizzare un messaggio.

Il seguente codice mostra un esempio di utilizzo del required:

```
<form name="myForm">
  <input name="myInput" ng-model="myInput" required>
</form>

<p>The input's valid state is:</p>
<h1>{{myForm.myInput.$valid}}</h1>
```



Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

5. HTTP Service

Il servizio $http di AngularJS è un servizio integrato che consente di effettuare richieste HTTP verso un server remoto. Questo servizio permette di inviare richieste GET, POST, PUT, DELETE e altre tipologie di richieste HTTP.

Il servizio $http può essere utilizzato all'interno di un controller o di una direttiva AngularJS, e viene utilizzato per recuperare dati da un server o per inviare dati ad esso.

Ad esempio, se volessimo recuperare un elenco di utenti dal server, potremmo utilizzare il servizio $http per effettuare una richiesta GET al server:

```
$http.get('/api/users').then(function(response) {
  $scope.users = response.data;
});
```

In questo esempio, abbiamo utilizzato il metodo $http.get() per effettuare una richiesta GET all'URL "/api/users". Il metodo .then() viene utilizzato per gestire la risposta della richiesta, ovvero l'elenco degli utenti che viene restituito dal server. La risposta viene assegnata alla variabile $scope.users all'interno del controller.

Il servizio $http supporta anche altre opzioni per la gestione delle richieste, come l'inclusione di parametri nella richiesta, l'invio di dati nella richiesta, la gestione degli errori e altro ancora.

Ecco una breve descrizione di ciascuna delle proprietà che compongono la risposta dal server:

- **.config**: l'oggetto di configurazione utilizzato per generare la richiesta HTTP. Questo oggetto contiene informazioni come l'URL richiesto, i parametri di richiesta e altre opzioni di configurazione.
- **.data**: la risposta dal server, che può essere una stringa o un oggetto. Il formato di questa risposta può variare in base all'endpoint del server.
- **.headers**: una funzione che consente di accedere alle informazioni sull'intestazione della risposta HTTP. Questo può includere informazioni come il tipo di contenuto, la codifica dei caratteri e altre informazioni utili.
- **.status**: il codice di stato HTTP della risposta dal server. Questo può essere un valore numerico come "200" (OK), "404" (non trovato) o "500" (errore del server).
- **.statusText**: una stringa descrittiva dello stato HTTP. Questo può essere una descrizione testuale del codice di stato, come ad esempio "OK" o "Non trovato".

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

Queste proprietà sono molto utili per gestire la risposta dal server e assicurarsi che la richiesta HTTP sia stata completata correttamente. In particolare, la proprietà ".data" contiene il corpo della risposta dal server, che può essere un oggetto JSON o una stringa, a seconda dell'endpoint del server.

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

Bibliografia

- V. Roberto, M. Frailis, A. Gugliotta, P. Omero. Introduzione alle Tecnologie Web, McGraw-Hill, 2010.
- A. S. Tanembaum. Computer Networks (4 ed.). Prentice Hall (4 ed.), 2003.
- Marco Andreini, Patrizia Andronico, Arianna Del Soldato, Cristian Lucchesi, Maurizio Martinelli, Giuseppe Vasarelli. Manuale corso di Tecnologie Web Istituto di Informatica e Telematica del CNR, 2005.

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

Sitografia

- HTML 4.01 Specification: http://www.w3.org/TR/html401/
- HTML5: http://www.w3.org/TR/html5/
- Javascript: JavaScript Tutorial (w3schools.com)
- HTML: http://www.w3schools.com/html/default.asp
- CSS: https://www.w3schools.com/css/default.asp
- CSS: http://www.w3.org/TR/CSS/
- NODEJS: https://nodejs.org

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).